

Dividing Head Controller — Wiring Guide (Arduino Nano)

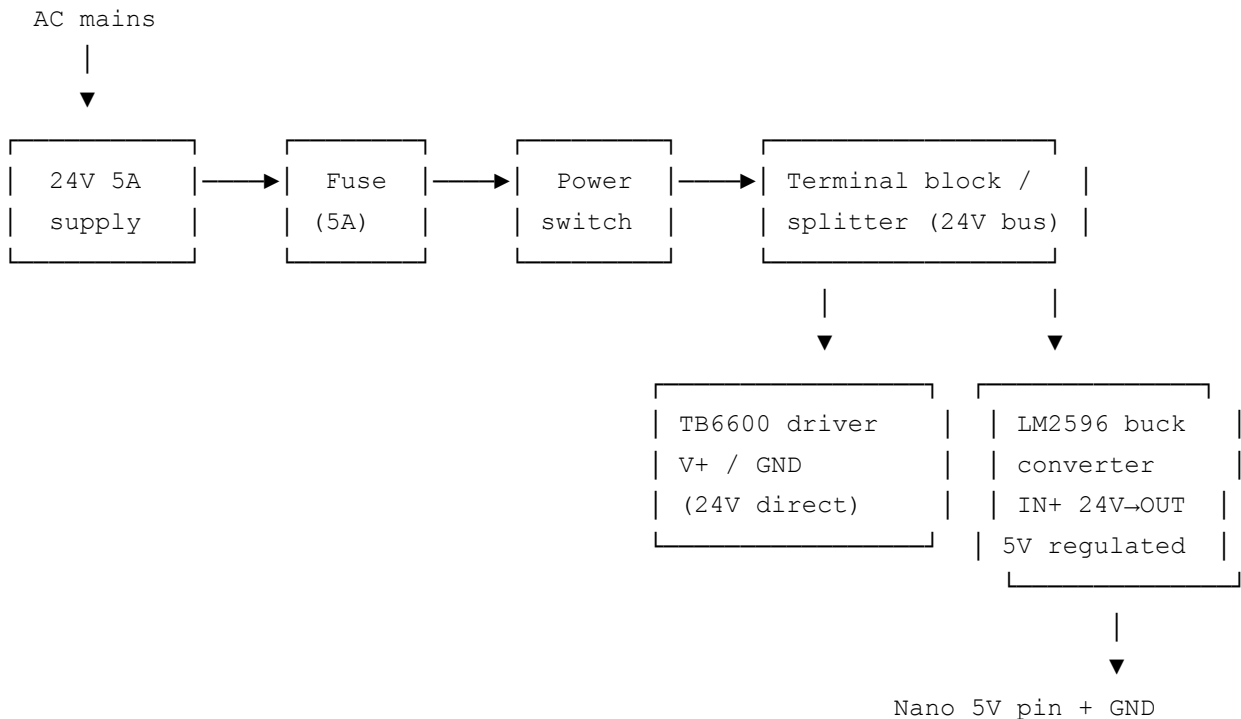
Important note on the K11

The VEVOR K11-100 dividing head is a mechanical unit + NEMA23 4-lead stepper motor with a 6:1 belt reduction. **It does not include a driver.** The Arduino sends STEP/DIR/ENABLE logic signals to a separate external stepper driver (TB6600, DM542, etc.), and that driver supplies the actual motor current to the K11's NEMA23 motor.

```
Arduino —STEP/DIR/EN→ Stepper Driver —A+/A-/B+/B-→ K11 NEMA23 m
(logic only)                (does the power)
```

Shared power supply architecture

A single 24V supply powers both the stepper driver (directly) and the Arduino Nano (through a buck converter, since the Nano's onboard regulator isn't meant to drop 24V down to 5V continuously). The 24V rail also has a fuse and power switch ahead of everything else.

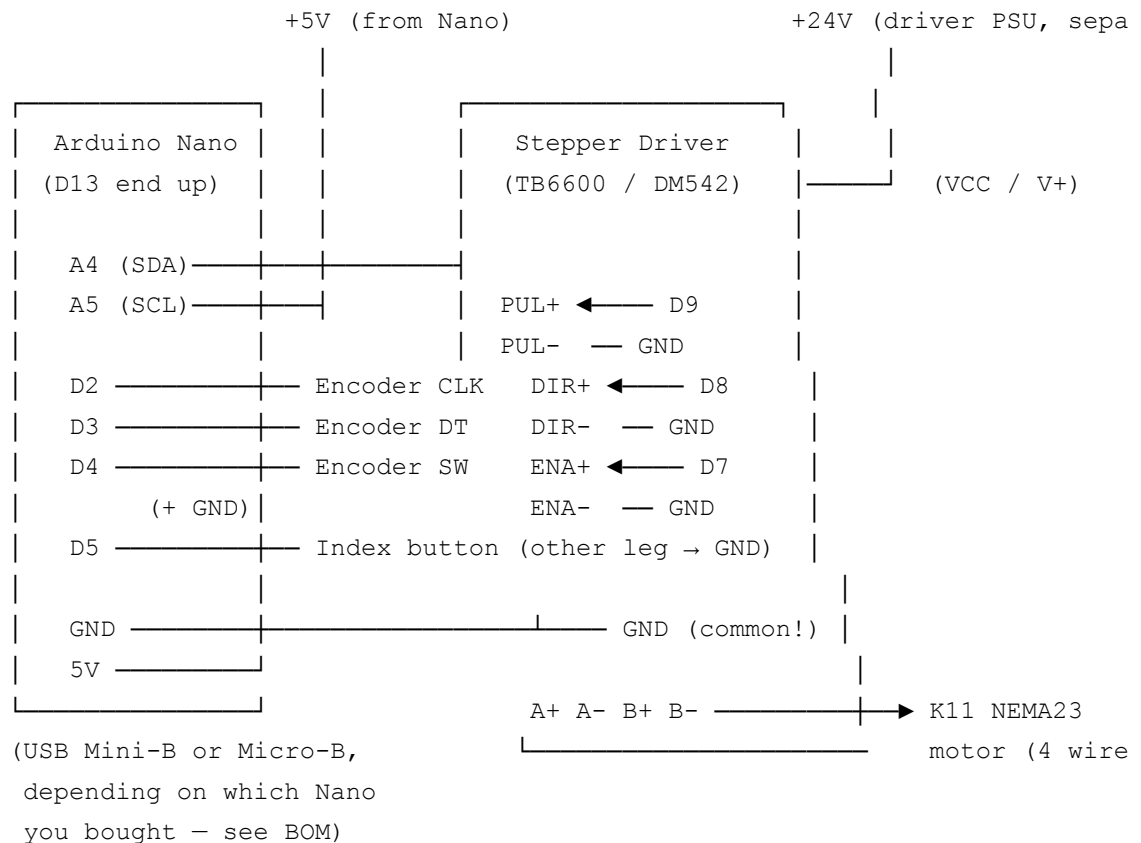


(also powers OLED,
encoder via Nano's 5V)

Why not just power the Nano from the driver's 24V or from VIN directly? The Nano's onboard linear regulator is rated for the board's own current draw only (a few hundred mA) and is only meant to take up to about 12V comfortably — running it continuously from 24V would waste a lot of power as heat and risks overheating the regulator. A small buck converter set to 5V and wired straight into the Nano's 5V pin (bypassing the onboard regulator entirely) is the standard, safe way to share one supply between logic and power electronics.

Fuse the 24V input, not each branch separately — one fuse sized for the combined worst-case draw (motor + electronics) ahead of the splitter protects the whole system; that's simpler and just as safe as fusing each branch for a project this size.

Block diagram



OLED (SSD1306 128x64, I2C):

VCC → 5V (or 3.3V — check your module's silkscreen)

GND → GND

SCL → A5

SDA → A4

Pin table

Signal	Arduino Nano pin	Notes
OLED SDA	A4	I2C data
OLED SCL	A5	I2C clock
OLED VCC	5V	some modules want 3.3V — check silkscreen
OLED GND	GND	
Encoder CLK	D2	must be interrupt-capable (D2 or D3)
Encoder DT	D3	
Encoder SW	D4	to GND when pressed, uses internal pull-up
Index button	D5	other leg to GND, uses internal pull-up
Driver STEP (PUL+)	D9	
Driver DIR+	D8	
Driver ENABLE+ (ENA+)	D7	LOW = enabled on most drivers — confirm on yours
Driver PUL-/DIR-/ENA-	GND	common ground with Nano
Nano 5V (power in)	buck converter OUT+	fed from the shared 24V rail, NOT USB, when running standalone
Nano GND (power in)	buck converter OUT-	common with 24V supply GND

Nano-specific notes

- **No mounting holes.** The standard Nano has no screw holes — it's meant to plug into a breadboard. The enclosure design uses side rails the board slides into rather than standoffs (see `enclosure_notes.md`).
- **USB connector varies by supplier.** The official Arduino Nano uses Mini-B; most

Amazon clones (CH340-based) use Micro-B. Check which one you bought before printing the enclosure's USB cutout position/size.

- **Power the Nano from the shared 24V supply via the buck converter's 5V output → Nano's "5V" pin, not USB.** This lets the whole system run from one wall supply. You can still plug in USB separately for reprogramming; the Nano automatically prioritizes whichever source is higher voltage at the moment (so leaving both connected is fine, and common practice for the Nano's diode-OR'd power inputs — but don't power the buck converter's 5V output back INTO the USB port itself, only into the 5V pin).
- **Uploading:** most clone Nanos need Tools → Processor → "ATmega328P (Old Bootloader)" in the Arduino IDE, or the upload will fail with a "programmer not responding" error.

Setting up the LM2596 buck converter (24V → 5V)

1. Before connecting it to the Nano, power the LM2596 module alone from the 24V supply and measure its output with a multimeter.
2. Turn the onboard potentiometer (small screwdriver) until the output reads as close to **5.00V** as you can get it. Recheck after a few minutes — these modules can drift slightly as they warm up.
3. Only then connect OUT+ to the Nano's 5V pin and OUT– to GND. Getting the voltage right *before* connecting it protects the Nano — feeding it much more than 5V through the 5V pin bypasses the onboard regulator's protection entirely.
4. Common ground: the buck converter's input GND, output GND, the Nano's GND, and the stepper driver's GND must all be tied together.

Critical wiring notes

1. **Common ground is mandatory.** Arduino GND, the driver's signal-side GND, the buck converter's GND, and the 24V power supply's negative terminal must all be tied together, or STEP/DIR pulses will be unreliable or won't register.
2. **The NEMA23 motor draws its current from the driver's 24V rail, not through the Arduino.** The Arduino only ever sees logic-level STEP/DIR/ENABLE signals.
3. **Set the driver DIP switches** to match the motor's rated current (check the plate on your K11's NEMA23 — commonly 2.5–3A) and your chosen microstepping (default in the firmware is 8x — change it in the Calibrate menu if you use a different setting).
4. **Driver ENABLE polarity** varies by model — most TB6600/DM542 boards enable the

outputs when ENA is pulled LOW, which is what the sketch assumes. If your motor is unexpectedly always “loose” or always locked, try flipping the `digitalWrite(PIN_ENABLE, ...)` logic.

5. Keep the 4-wire motor cable separate from the STEP/DIR signal wiring where practical (don’t bundle them together) to reduce electrical noise on the pulse lines.
6. **Add a bulk capacitor across the 24V rail near the driver** (1000–2200μF, rated ≥35V) if one isn’t already built into your driver board or power supply. Steppers generate voltage spikes (back-EMF) when decelerating that a small supply can’t absorb on its own; this capacitor protects the driver and buck converter from those spikes. Many TB6600 boards include this on the PCB already — check yours before adding a second one.
7. **Fuse the 24V input** (see BOM) between the power supply and everything else, sized to the combined worst-case current draw of the driver + buck converter + margin (a 5A fuse comfortably covers a single NEMA23 stepper plus the Nano/OLED/encoder electronics).
8. **Switch on the low-voltage (24V DC) side**, not the mains side — it’s safer to wire, and a DC-rated toggle/rocker switch is cheap and widely available (see BOM).

Steps-per-degree math (for reference)

```
steps_per_degree = (motor_steps_per_rev × microstep × gear_ratio) / 360
```

Defaults in firmware: 200 steps/rev × 8 microstep × 6.0 gear ratio ÷ 360 = **26.67**

steps/degree. If you change the driver’s microstepping DIP switches, update the “Driver microstep” value in the Calibrate menu to match — the firmware recalculates automatically.